AI in images and video processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Spring 2025

Feature Extraction & Segmentation

Session 4

Feature Extraction & Segmentation

Session 4

Objectives:

- Understand what feature extraction is and why it is important
- Identify different types of image features (edges, texture, color, shape)
- Explain the difference between local and global features
- Apply popular feature extraction algorithms (SIFT, ORB) in practice
- Describe and implement common image segmentation techniques
- Analyze and compare feature extraction and segmentation results

Introduction & Motivation

Why do we need features and segmentation in computer vision?

Introduction & Motivation

Why do we need features and segmentation in computer vision?

- Raw pixel values are not enough for most computer vision tasks.
- **Feature extraction** transforms images into numerical representations that a model can understand (**edges, textures, color, shapes**).
- **Segmentation isolates regions of interest** (e.g., separate an object from the background), making it easier to analyze or classify images.
- **These processes are foundational for tasks like recognition, tracking, medical analysis, and autonomous vehicles.**

Feature extraction

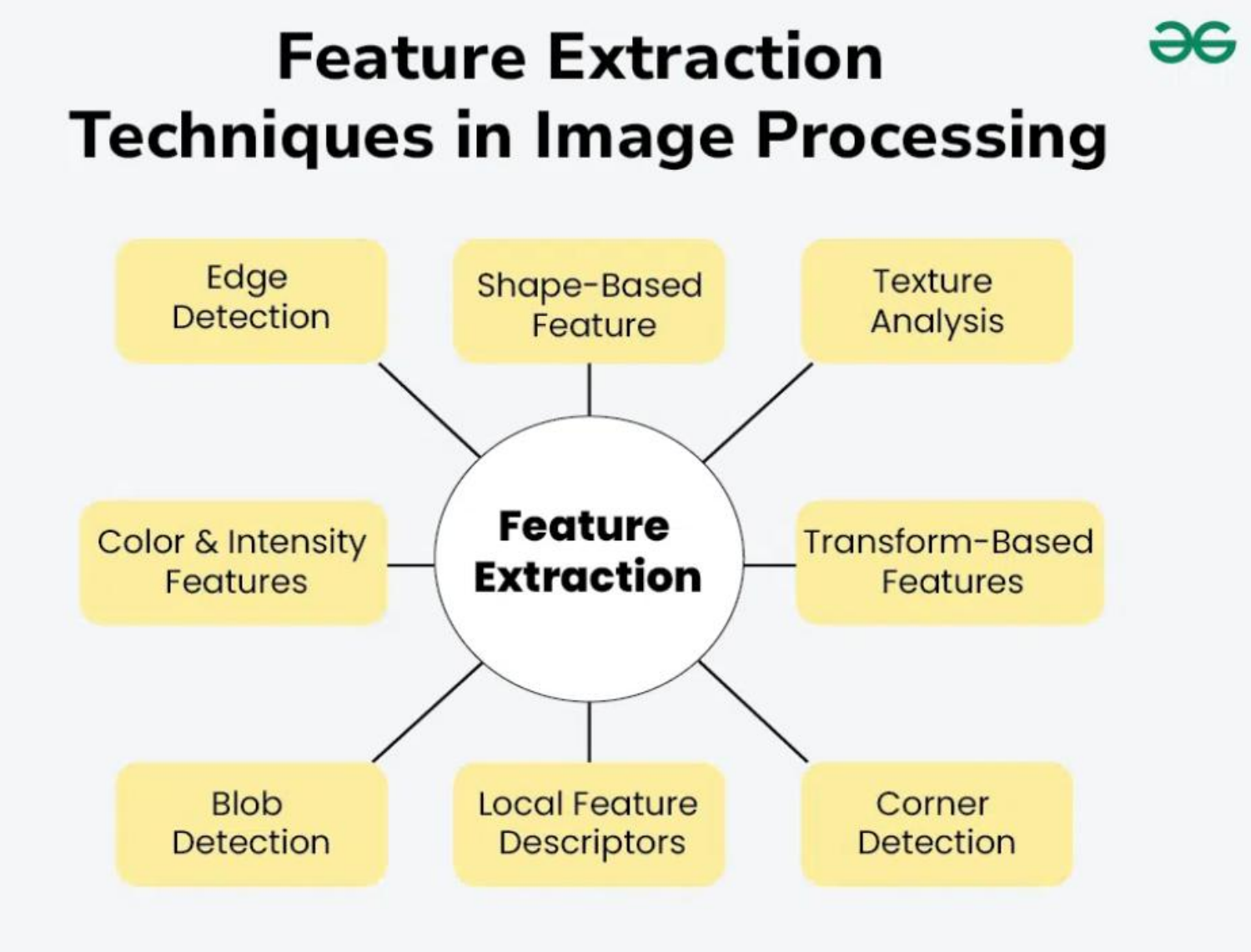
Definition: Identify important characteristics or properties of the segmented regions or the entire image. This process transforms raw data into a set of features that can be used for further analysis or machine learning. **However no need to extract features when using CNN (only for RF, SVM, etc..)**

Purpose: Reduces the complexity of data to improves performance of learning algorithms (classification, recognition, and regression)

Type of features:

- **Edge Features:** Sudden changes in intensity, highlight object boundaries
- **Texture Features:** Repeated local intensity variations, capture material or surface type
- **Color Features:** Distribution of colors (RGB, HSV), useful for classification/tracking
- **Shape Features:** Object outlines, contours, geometric moments, helpful for object recognition

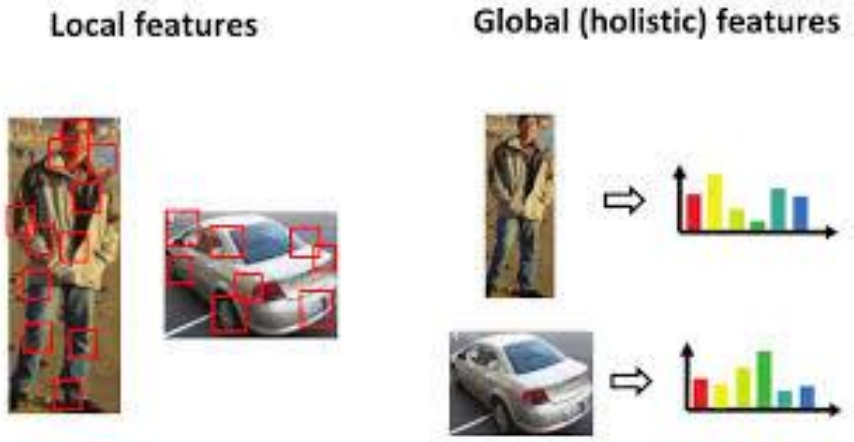
Feature extraction



Local vs. Global Features

Local Features: Calculated in small neighborhoods; robust to image changes

- SIFT, corners, blobs
- Matching and recognition



Global Features: Describe entire image; fast, less robust to change

- Color histogram, Gabor texture
- Image retrieval, scene classification

In Practice:

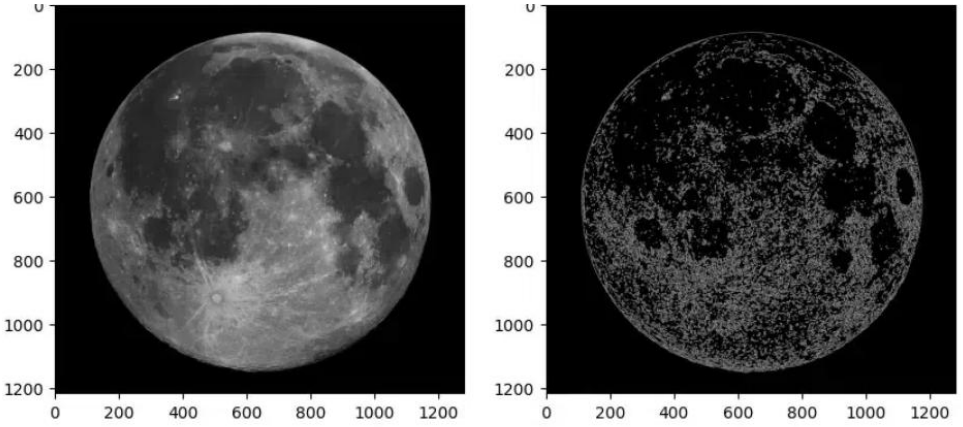
- Use local features for detailed tasks (e.g., matching parts of two images)
- Use global features for broad similarity (e.g., find "all sunset images")

Edge detection (session 2)

Definition: Edges: Locations with strong intensity change.

Common detectors: Sobel, Canny

Used for: object boundary detection



Edge Detection using OpenCV

```
img = cv2.imread('moon.jpg',cv2.IMREAD_GRAYSCALE)

# Apply Canny edge detector
edges = cv2.Canny(img, 100, 200)
```

Harris corner detection

Purpose: Detect points in an image where the intensity changes in two directions — i.e., corners.

Why corners?
Corners are highly distinctive and stable under transformations, making them excellent for matching and tracking.

Algorithm Overview:

- Compute gradients in x and y.
- For each pixel, build a matrix from the gradients over a window.
- Calculate “cornerness” (R value, **High R**: Likely a corner.).
- Threshold R to detect corners.

Texture detection

What is Texture?
Texture is the “feel” of an image surface—patterns, granularity, and repeated structures that make it possible to distinguish materials like grass, sand, brick, or fur, even if their colors are similar.

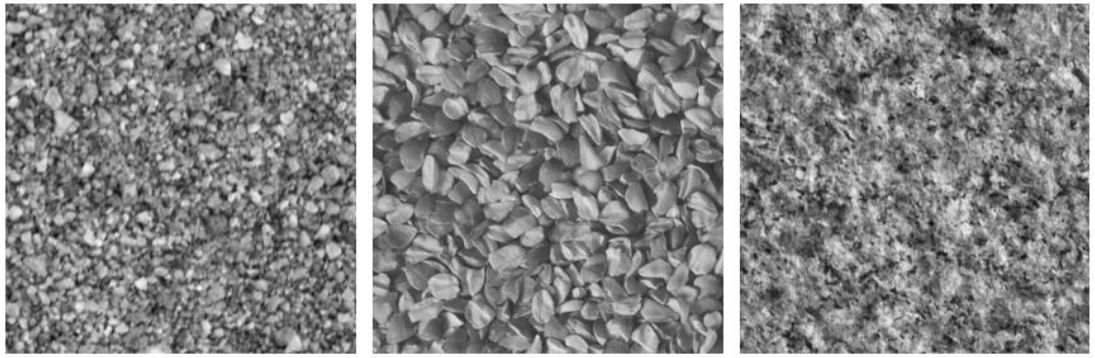
Why is Texture Important?
Our eyes use texture all the time: - Is that a smooth desk or a rough stone? Are you looking at grass or carpet, sand or concrete?
Texture helps us answer these questions visually, and the same goes for machines!

How Do We Capture Texture in Images?
Raw pixels don’t say much about texture. Instead, we need to analyze how intensities vary locally:

- Is there a repeated pattern?
- Are there rapid or slow changes in brightness?
- Does the pattern stay the same across a region?

Applications of Texture Analysis:

- Medical imaging: distinguish healthy and diseased tissue
- Industrial inspection: identify defects on surfaces
- Remote sensing: classify forests, water, and urban areas
- Face recognition, fingerprint analysis, document analysis



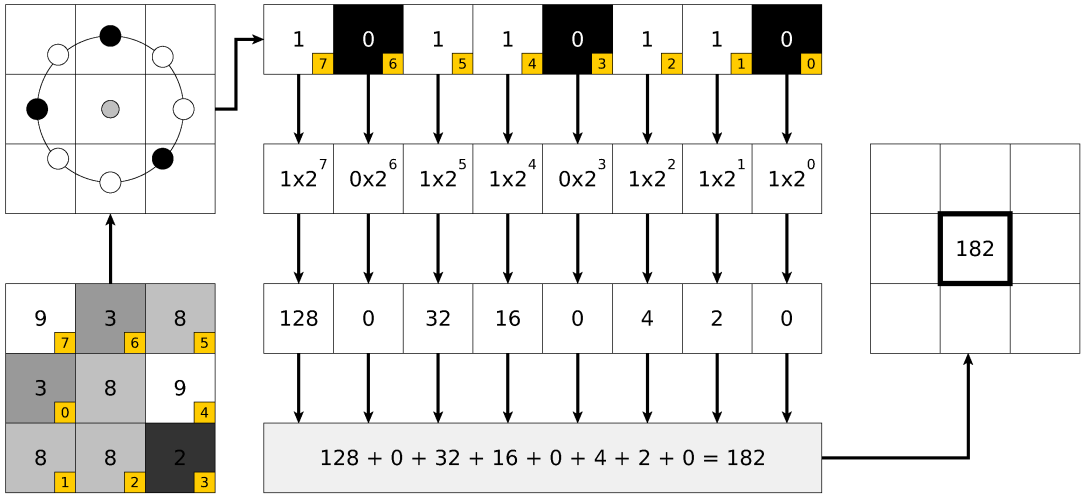
Texture detection

Methods:

- **Local Binary Patterns (LBP):** Compares each pixel with its neighbors to create a simple code—fast and robust.
- **Gabor Filters:** Analyze textures at different scales and orientations (like tuning to different “frequencies”).
- **Haralick Features:** Capture more complex statistics of texture.

Local Binary Patterns (LBP):

Purpose: Simple, fast way to describe local texture. Robust to illumination changes, excellent for material/surface recognition.



How it works:

- For each pixel, compare its value with its 8 neighbors.
- If a neighbor is greater or equal, mark “1”; else, “0”.
- Read the 8 bits as a binary number → LBP code.
- Every pixel is replaced by its LBP code.
- Extract features from the histograms

Applications:

Face recognition, defect detection, medical imaging, surface inspection.

Variants

Multi-scale LBP

Rotation-Invariant LBP

Address scale and rotation issues.

Local Binary Patterns (LBP):

In Python local_binary_pattern(img, P, R, METHOD)

- **P:** Number of circularly symmetric neighbor set points (typically 8)
- **R:** Radius of circle (1 pixel)
- **METHOD:**
  - **default:** Original local binary pattern which is grayscale invariant but not rotation invariant.
  - **ror:** Extension of default pattern which is grayscale invariant and rotation invariant.
  - **uniform:** Uniform pattern which is grayscale invariant and rotation invariant, offering finer quantization of the angular space. For details
  - **nri_uniform:** Variant of uniform pattern which is grayscale invariant but not rotation invariant.
  - **var:** Variance of local image texture (related to contrast) which is rotation invariant but not grayscale invariant.

Haralick Features:

Haralick features are a family of texture descriptors based on the Gray-Level Co-occurrence Matrix (GLCM), introduced by Robert Haralick in 1973.

GLCM: Measures how often pairs of pixels with specific values (gray levels) occur at a certain spatial relationship (distance and angle) in an image.

Key Haralick Features:

- **Contrast** — Difference between highest and lowest intensity pairs (how much variation)
  - Higher values mean more variation; e.g. gravel should have higher contrast.
- **Correlation** — How correlated a pixel is to its neighbor (regularity, patterns)
  - High values mean the pattern is more regular/predictable—bricks > grass > gravel.
- **Energy** — Sum of squared elements in GLCM (uniformity)
  - Higher for uniform, repeating patterns (wall > grass/gravel)
- **Homogeneity** — How close GLCM elements are to its diagonal (local similarity)
  - Higher for textures where neighboring pixels are similar (wall/grass > gravel).

Haralick Features:

Haralick features are a family of texture descriptors based on the Gray-Level Co-occurrence Matrix (GLCM), introduced by Robert Haralick in 1973.

GLCM: Measures how often pairs of pixels with specific values (gray levels) occur at a certain spatial relationship (distance and angle) in an image.

Key Haralick Features:

- **Contrast** — Difference between highest and lowest intensity pairs (how much variation)
  - Higher values mean more variation; e.g. gravel should have higher contrast.
- **Correlation** — How correlated a pixel is to its neighbor (regularity, patterns)
  - High values mean the pattern is more regular/predictable—bricks > grass > gravel.
- **Energy** — Sum of squared elements in GLCM (uniformity)
  - Higher for uniform, repeating patterns (wall > grass/gravel)
- **Homogeneity** — How close GLCM elements are to its diagonal (local similarity)
  - Higher for textures where neighboring pixels are similar (wall/grass > gravel).

Gabor filter:

What is a Gabor Kernel?
A Gabor kernel is a type of bandpass filter, i.e. it passes frequencies in a certain band and attenuates the other frequencies outside such band.

Mathematically:
A Gabor kernel is a Gaussian envelope (which controls the size and shape) multiplied by a sinusoidal wave (which controls frequency and orientation).

How It Works:
Gaussian envelope: - Defines the spatial support ("window") of the filter
- Controls size, shape, and spatial locality
Sinusoidal wave: - Sets the frequency (how fine/coarse the patterns the filter detects)

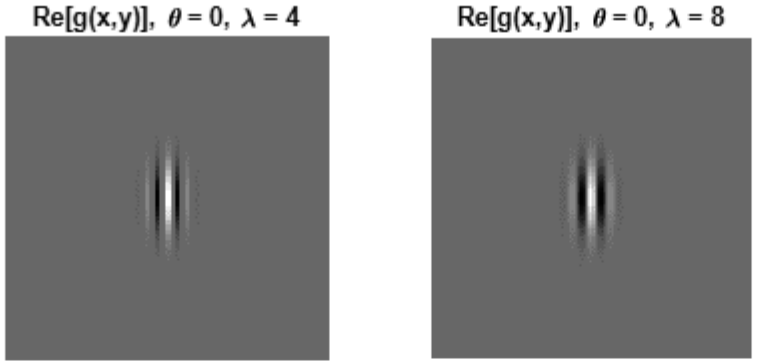
Orientation: Rotating the sinusoid makes the filter sensitive to patterns at different angles

What Does This Mean for Images?
The Gabor filter is sensitive to texture at a specific scale (frequency) and direction (orientation). It acts as a "texture microscope," highlighting image features that repeat at particular angles or spacings.

Gabor filter:

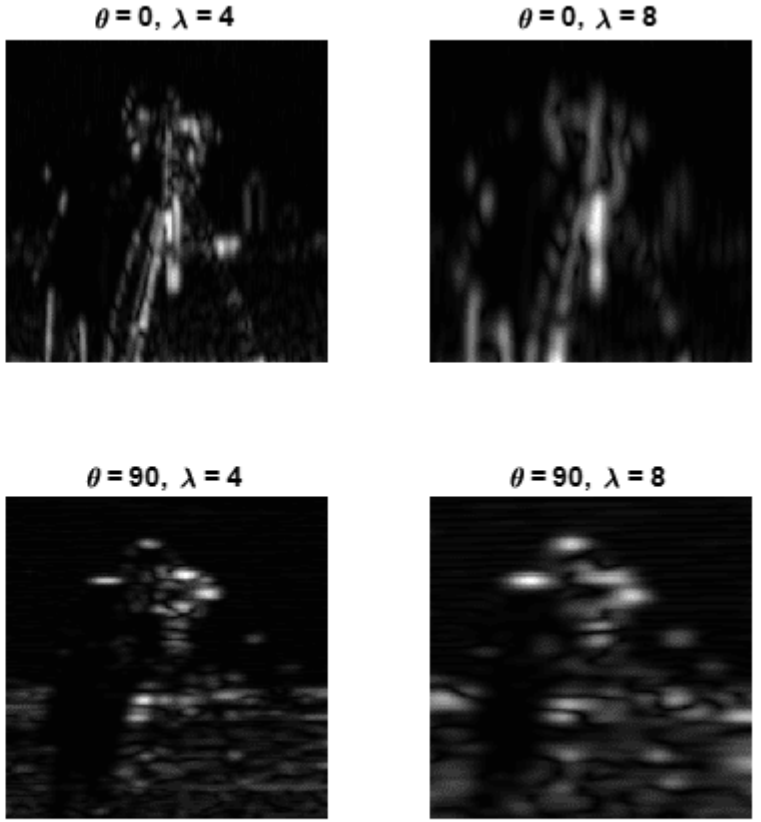
Original image

*



Gabor filters

=



Filtered images

Horizontal and vertical features are extracted with theta=0 and theta=90. Finer details with lower wavelength values.

Gabor filter:

In Python local_binary_pattern(img, P, R, METHOD)

- **P:** Number of circularly symmetric neighbor set points (typically 8)
- **R:** Radius of circle (1 pixel)
- **METHOD:**
  - **default:** Original local binary pattern which is grayscale invariant but not rotation invariant.
  - **ror:** Extension of default pattern which is grayscale invariant and rotation invariant.
  - **uniform:** Uniform pattern which is grayscale invariant and rotation invariant, offering finer quantization of the angular space. For details
  - **nri_uniform:** Variant of uniform pattern which is grayscale invariant but not rotation invariant.
  - **var:** Variance of local image texture (related to contrast) which is rotation invariant but not grayscale invariant.

Color features

What is color feature?

Color features describe the distribution of color information in an image, helping machines recognize, compare, and classify scenes or objects based on their color content.

Why is Texture Important?

Many objects/scenes are easily distinguished by color—think of fruit, skies, traffic lights, etc.

Method?

- Color histogram
- Color moments
- Color space

Color histogram

What is a Color Histogram?
A color histogram represents the distribution of pixel colors in an image.
It shows how many pixels fall into each range (bin) of color intensity for one or more color channels.

Can be computed per **channel (R, G, B)**, or in other **color spaces (HSV, Lab)**.

How Is It Computed?

- Divide the possible pixel values (0–255 for 8-bit images) into bins.
- Count the number of pixels whose value falls into each bin

Why Use Color Histograms?

- Scene Recognition:
  - Quickly tells you if an image is “mostly blue” (ocean/sky), “mostly green” (forest/field), or “mostly red” (sunset/flowers).
- Image Retrieval:
  - Find images with similar color “profiles” even if the content is different.
- Tracking/Detection:
    - Track colored objects (e.g., a red ball in sports footage).

Color moment

Color moments are statistical summaries:

- **Mean:** The average color value per channel (B, G, R).
- **Standard deviation:** How much the color varies (spread).
- **Skewness:** Whether the color is biased toward low or high values (asymmetry).
  - Skewness ≈ 0: The histogram is fairly symmetric.
  - Skewness > 0: The distribution has a longer right tail (few high-intensity pixels).
  - Skewness < 0: The distribution has a longer left tail (few low-intensity pixels)

Motivation:

- **Compact feature:** Only a few numbers per image.
- **Used for:** Fast image retrieval, scene comparison, input to ML models.

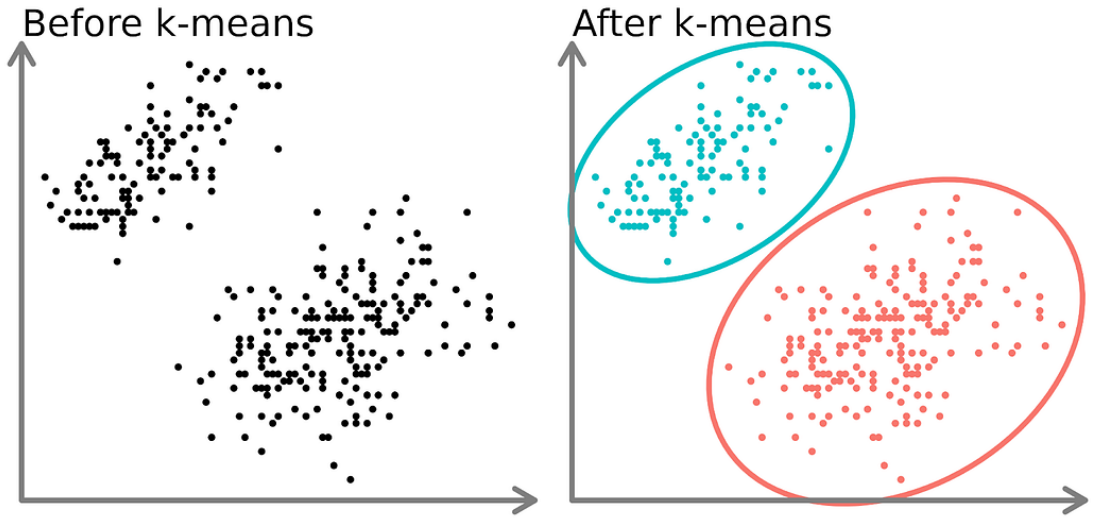
Dominant Colors with K-means Clustering

What is K-means?

An unsupervised machine learning algorithm that groups data points (e.g., pixels) into K clusters based on their similarity.

How does it work?

- Choose K (number of clusters).
- Randomly initialize K cluster centers (centroids).
- Assign each pixel to the nearest
- Recompute centroids as the mean of assigned pixels.
- Repeat steps 3–4 until assignments stabilize.



Limitations:

- Must choose K in advance (try the “elbow method”).
- Sensitive to initial centroids.
- May struggle with complex or non-spherical clusters.

Shape features

Shape features describe the geometric properties of objects or regions in an image and help recognize, classify, and analyze objects regardless of their color or texture.

The two most common tools:

Contours: Boundaries of shapes/objects (see session 2).

Moments: Mathematical properties:

- **Area**: Size of the shape.
- **Centroid**: Center point.
- **Orientation/Eccentricity**: Main direction or “tilt”.
- **Hu Moments**: Set of 7 values that are invariant to translation, rotation, and scale (good for matching shapes)

Feature extractions

There are various techniques and algorithms used for feature detection and extraction. One popular approach is the use of **local feature detectors**, such as the **Scale-Invariant Feature Transform (SIFT), Speeded-Up Robust Features (SURF), and Oriented FAST and Rotated BRIEF (ORB)**. These methods aim to identify and extract features that are **invariant to changes in scale, rotation, and illumination.**

Descriptor	Invariant to	What for?	Note
SIFT	Scale, rotation	Keypoint matching, stitching	Patented (not for commercial use in old OpenCV, now patent expired)
SURF	Scale, rotation	Fast detection	Patent/licensing issues
ORB	Rotation	Real-time, patent-free	Fast, good for robotics
BRIEF	None (very fast)	Fast matching	Used with FAST keypoints
HOG	Illumination	Pedestrian/vehicle detection	Used in object detectors

Feature extractions

What is a Keypoint Detector & Descriptor?

- **Keypoint detector:** Finds “interesting” locations in an image (corners, blobs, etc.).
- **Descriptor**: For each keypoint, summarizes the appearance of the neighborhood in a vector.

Goal: Describe local regions so they can be matched between different images (even with changes in scale, rotation, or lighting). Most of the methods aim to identify and extract features that are **invariant to changes in scale, rotation, and illumination.**

Descriptor	Invariant to	What for?	Note
SIFT	Scale, rotation	Keypoint matching, stitching	Patented (not for commercial use in old OpenCV, now patent expired)
SURF	Scale, rotation	Fast detection	Patent/licensing issues
ORB	Rotation	Real-time, patent-free	Fast, good for robotics
BRIEF	None (very fast)	Fast matching	Used with FAST keypoints
HOG	Illumination	Pedestrian/vehicle detection	Used in object detectors

SIFT: Scale-Invariant Feature Transform

Key properties:

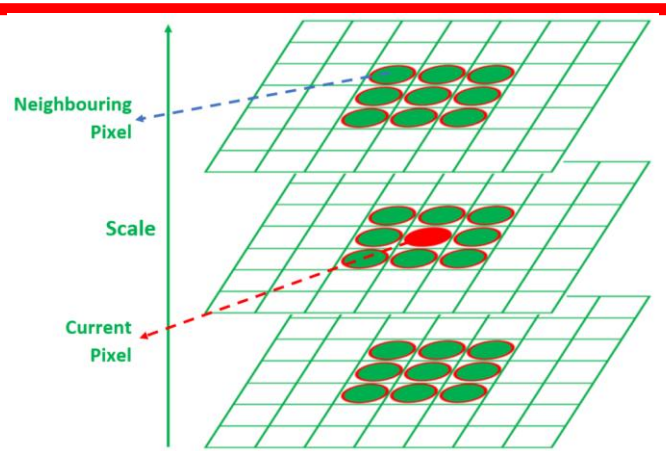
- Invariant to scale and rotation
- Robust to changes in illumination and small distortions

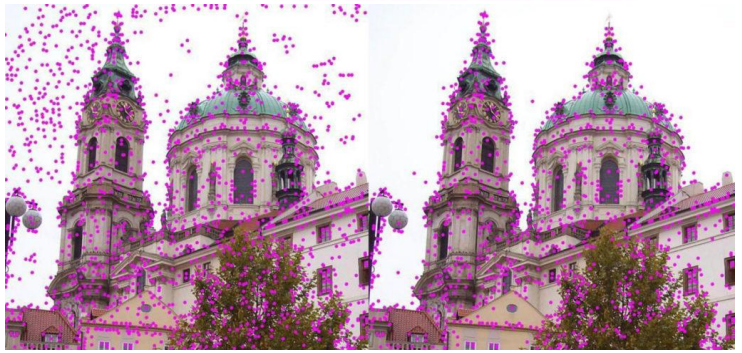
Process:

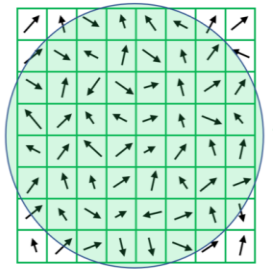
- Scale space peak detection: Scale space means making many blurred versions of the image, each one blurrier than the last. This is done by applying Gaussian filters with different amounts of blur**. look at the image in different "zoom levels"—from fine details to bigger structures.**

- Key Point Localization: refinement of keypoints selected in the previous stage using the difference of Gaussians (DoG) to measure how “strong” or “sharp” the keypoint is. (DoG: Blur the image twice, and substract)

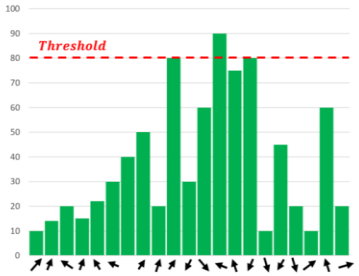
- Assigning Orientation to Keypoints to ensure detection which is invariant. For this, computes a descriptor for each keypoint by considering the local gradient information in its neighborhood**.**







Assigning Gradient Magnitude /Direction to neighbourhood



Allocating orientations to histogram

ORB : Oriented FAST and Rotated BRIEF

What is ORB?

ORB is rotation invariant and resistant to noise, making it suitable for images with varying orientations and reliable in real-world scenarios with noisy images. It combines two algorithms: FAST (for keypoint detection) and BRIEF (for descriptor extraction)

Process:

1. FAST: Keypoint Detection: For each pixel, check if a circle of neighboring pixels are all brighter/darker than the center.
    - If yes, it's a corner (keypoint).

2. Harris Score for Ranking: ORB ranks FAST corners using a Harris corner measure and keeps the top N strongest points.

3. Orientation Assignment: For each keypoint, compute the intensity centroid in its neighborhood.
    - The direction from the keypoint to this centroid gives its orientation.

4. BRIEF: Binary Robust Independent Elementary Features: generates a binary string for each patch:
Randomly selects pairs of pixels within the patch.
For each pair: if the first is brighter than the second, output a 1, else a 0.

5. Descriptor Matching: ORB descriptors are matched using Hamming distance (counting bit differences).

Histogram of Oriented Gradients (HOG)

What is HOG?

HOG is a feature descriptor used in computer vision and image processing to detect objects by capturing edge and gradient structures.

- Converts image regions into histograms of gradient orientations.
- Used in: pedestrian detection, face detection, car detection, etc.

Process:

1. **Convert the image to grayscale** and apply normalization
2. **Gradient Computation**: Calculate the gradient and magnitude of the image using techniques like Sobel operators.

$$G_x = I(x+1, y) - I(x-1, y), \quad G_y = I(x, y+1) - I(x, y-1)$$

$$\text{Magnitude: } M = \sqrt{G_x^2 + G_y^2}, \quad \text{Orientation: } \theta = \arctan \frac{G_y}{G_x}$$

3. **Cell division:** Divide the image into small cells (e.g., 8x8 pixels) to capture local gradient information.
4. **Orientation Histograms:** For each cell:

- Create a histogram of gradient orientations.
- Each pixel votes for a bin using its magnitude.

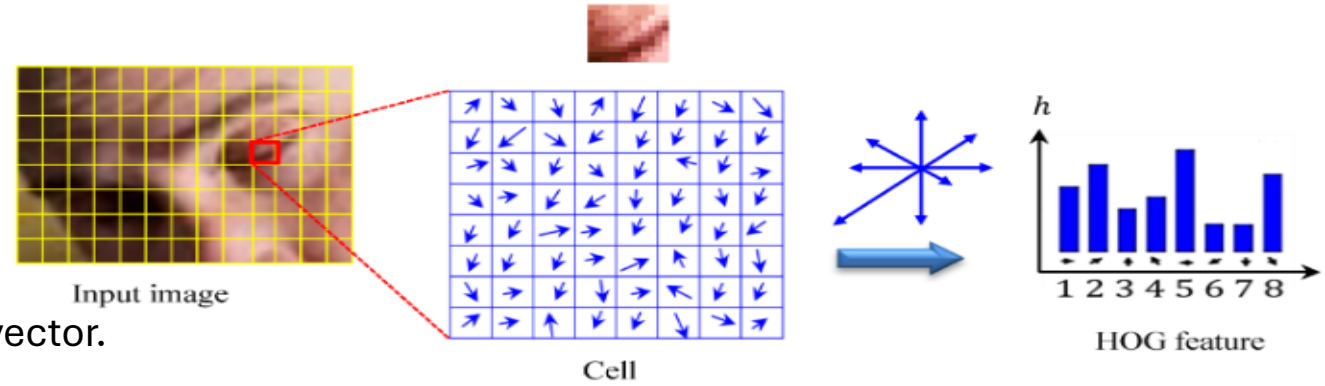
5. Block Normalization:

- Group cells into blocks (e.g., 2x2 cells).
- Normalize the histograms in each block

to reduce lighting/shadow effects

5. Normalized vector Feature Vector Construction:

Concatenate all normalized histograms into one long feature vector.



Segmentation

Definition: Process of partitioning an image into multiple segments or regions without prior labeling.

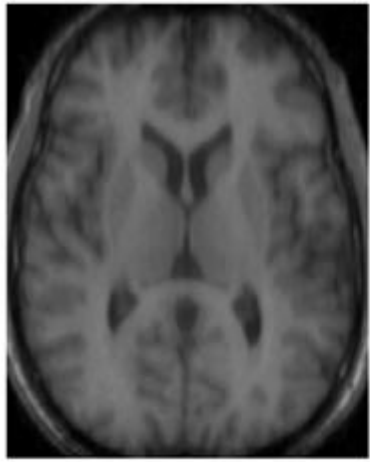
Purpose: The main goal is to identify and isolate objects or regions within an image (object detection, image analysis, and scene understanding or to separate foreground/background)

Methods: Common techniques include:

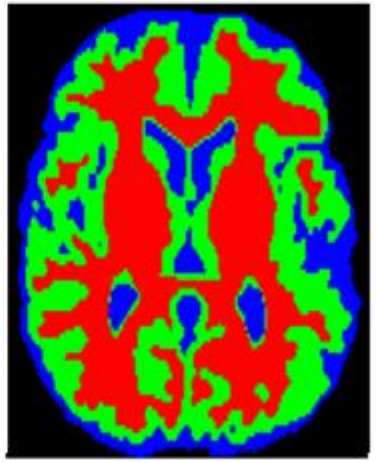
- Thresholding: Simple, based on pixel intensity
- Clustering (K-means): based on pixels color/intensity
- Watershed: Splits touching/overlapping objects.

Output: The result of segmentation is typically a set of labeled regions or segments within the image, where each segment corresponds to a distinct object or area of interest.

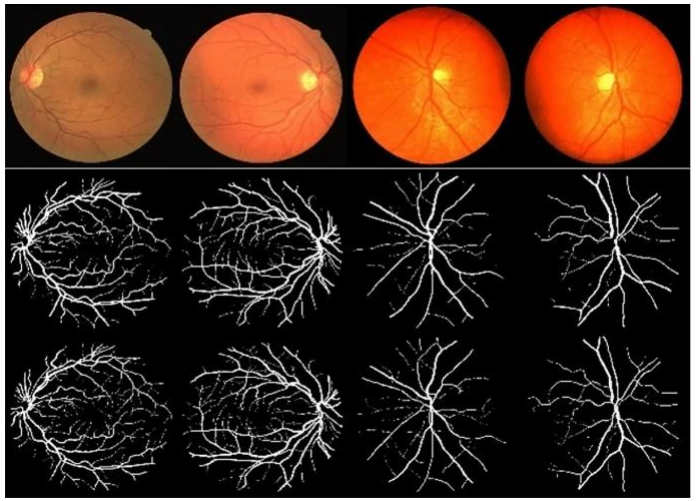
Segmentation



(a) coupe axial



(b) segmentation des tissus



Segmentation: Thresholding

Goal: Separate foreground (object) from background based on intensity (pixel value).

How:

- Choose a threshold value T.
- Any pixel value > T becomes white (foreground/object).
- Any pixel value ≤ T becomes black (background).

Variants:

- Global Threshold: Same T for the whole image (simple, fast).
- Adaptive Threshold: T can vary locally (good for uneven lighting).
- Otsu's Method: Algorithmically finds "best" T to separate two peaks in the image histogram.

Use Cases:

- Scanning/Document binarization
- Separating objects from simple backgrounds
- Preprocessing for contour detection

Original Image

Thresholded and segmented Image

Segmentation: K-means

Goal: The output is a “simplified” image, where each pixel takes on the color of its cluster center.

Workflow:

- Reshape the image into a list of pixels (each as a feature vector).
- Apply K-means to assign a cluster label to each pixel.
- Replace each pixel with its cluster center color.

Advantages:

- Works well for images with distinct color/texture regions.
- Useful for image “posterization” and pre-segmentation.

Limitations:

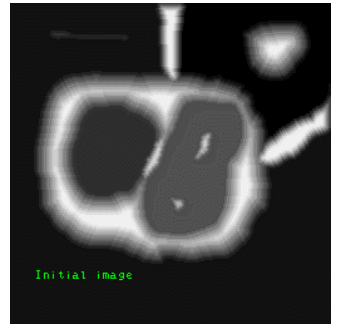
- Sensitive to K value (number of clusters).
- May not separate touching objects or handle uneven lighting well.

Typical Applications:

Color quantization, object/background separation, preprocessing

Segmentation: Watershed

What is the Watershed Algorithm?
A region-based segmentation technique, especially powerful for separating overlapping or touching objects.



Concept:

- Treat the grayscale image as a topographic surface (mountains/valleys).
- Now imagine filling each valley with water.
- To prevent water from different source to merge, we build a “barrier” (the watershed line)

Water “floods” from marked “seed” regions; when waters meet, boundaries (watersheds) are formed.
Requires foreground and background markers to work effectively.
Often used after thresholding/morphology to segment objects that touch.

Strengths:
Good at splitting touching/overlapping objects (e.g., coins, cells).
Weaknesses:
Needs good markers; sensitive to noise.

Segment Anything Model (SAM)

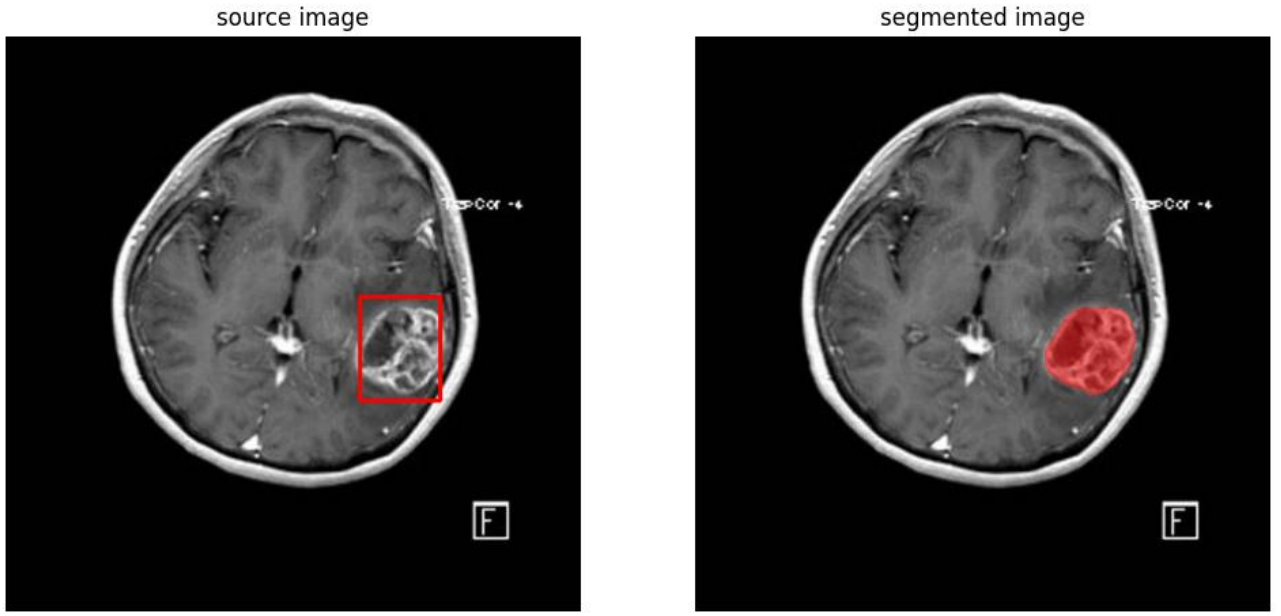
What is SAM?

- Deep learning model by Meta AI (2023)
- Designed to “segment anything” in any image, with little or no manual guidance

- Trained on 11 million images, 1.1 billion masks

How does it work?

- You can click, draw a box, or prompt text to tell it “what” to segment.
- SAM instantly returns a high-quality mask for the object or region.



[Segment Anything | Meta AI]

LAB

IA_images_Lab4_questions
(lab/lecture Features and Segmentation)